KERNELLENS: RECOVERING THE TRITON-TO-C++ ENGINE CONTRACT

Justin Duc [1]

ABSTRACT

OpenAI Triton has become the standard way to author custom GPU kernels. Thanks to its simplicity, this interface is generally preferred over CUDA, allowing one to write quickly, a characteristic that is essential in a research context. But this simplicity has a cost: there is no efficient way to use Triton kernels in standard inference engines such as TensorRT and ONNX Runtime out of the box. A normal pipeline would require rewriting the Triton kernels in CUDA, and programming many interfaces specialized for ORT (ONNX Runtime) and TRT (TensorRT). Despite the lack of tools for this task, the interface of Triton to a C++ engine is implicit and undocumented but recoverable automatically from the compiled artifacts, allowing a Triton kernel to be re-established as a native TensorRT and ONNX Runtime plugin. This is what our work, KernelLens, intends to prove through three recovery techniques: (i) parse the PTX entry signature to rebuild the host argument array that Triton's backend has re-bitwidth'd and pruned; (ii) capture the dynamic launch grid symbolically using SymInt and Python tracing, and transpile it to C++ that can be evaluated at runtime; (iii) classify each argument as input, output or in-place by analyzing the operations in the AST, even through nested helpers.

Establishing this interface inside the ONNX graph enables graph optimisation around the kernel while keeping the speedup provided by the kernels when graph fusing is not enough. Instead of $1{,}137$ lines of hand-written C++, only nine lines of Python are needed to attain the same result on an RMSNorm kernel from LigerKernel. In order for this project to stay up to date after release, we produce both V2 and V3 versions of the plugin, handling any version of TensorRT implicitly. KernelLens reproduces native Triton exactly on research workloads and on the fourteen module patterns; and on a block whose attention is a custom squared-ReLU flash kernel, it is $1.99\times$ faster than the folded-only path and $1.08\times$ faster than the unfused kernel path (eager Triton), while remaining bit-exact on two architectures (A100-SXM4-80GB and RTX 5060 Ti 16 GB), covering both `sm_80` and `sm_120`. We finish by showing results on real LLMs: replacing every decoder LayerNorm of OPT checkpoints from 125M to 6.7B, KernelLens TRT gives the lowest prefill latency at every size and precision.

1 INTRODUCTION

Inference engines such as ORT and TRT have become the standard way to serve models, and Triton the way to optimize models through kernel authoring. Despite the fact that those two technologies feel like they are meant to work hand in hand, they meet awkwardly at deployment. Triton, with `@triton.jit` functions, compiles the DSL to PTX and launches from Python, while inference engines (TensorRT and ONNX) both serve in C++ and apply whole-graph optimization, kernel selection and VRAM planning — but there exists no way for a model that calls a Triton kernel to be traced by an exporter and executed by those engines.

The common conclusion is that Triton and the engines are *incompatible*. We argue the opposite, and that is the thesis of this paper:

> A compiled Triton kernel is a complete self-contained artifact; the interface is implicit and undocumented but recoverable automatically.

Once recovered, the interface is established based on the inference engine's expected format: the kernel becomes a native custom operator and, because it is properly injected into the graph instead of breaking it, the engine is able to fold and tune the layers around it. The result keeps both the kernel optimization and the graph optimizations brought by the engines.

What is new. This paper doesn't present a new algorithm but rather a capability that did not previously exist, all enabled thanks to three main techniques:

(i) **PTX entry-signature recovery.** Triton's backend automatically re-bitwidths and drops arguments, so basing

[1]Galiad Research. Correspondence to: Justin Duc <justin.duc@galiad-research.com>.

Preprint, October 2026. Code and measurements: https://github.com/Just1truc/kernel_lens

the launch parameters on Python is a common error. We propose a method to parse the `.entry` declaration, map its slots back onto the kernel's arguments, and rebuild the host launch array, making it possible to launch Triton kernels through their PTX from custom C++ plugins.

(ii) **Symbolic launch-grid capture and C++ transpilation.** The grid is a Python lambda over runtime shapes, and knowing how many tiles need to be launched is essential. To solve this, we recover it with a dual-pass tracer and transpile it to C++ integer arithmetic that is evaluated at runtime, directly in the engine's custom plugin.

(iii) **Static AST argument taxonomy.** Classifying arguments as input or output cannot be done by heuristics, and determining which is which is essential to both ONNX and TensorRT, as they allocate outputs differently; an error in classification would crash the whole pipeline. We recover the classes through AST parsing, using Triton's loads and stores to memory to determine the class of each entry.

Contributions. Our contributions cover the following:

(1) The implementation of the recovery techniques, integrated into a compile pipeline that transforms a `torch.nn.Module` into an ONNX graph plus TensorRT and ONNX Runtime plugin libraries.

(2) A run pipeline that binds PyTorch GPU addresses directly to either of the engines with no copy.

(3) A complete empirical study spanning parity, coverage, folding plus kernels, latency, end-to-end models and quantization, in order to show that the composition is correct and that keeping the kernel plus folding is faster than either alone.

2 Related Work

2.1 GPU kernel languages

Triton [1] lowers a Python DSL through its own LLVM backend. It chooses tiling and vectorisation, but it is made to write kernels and cannot serve complex models by itself: it is a language that allows kernel authoring. Halide [12] allows one to experiment with scheduling easily, but it doesn't offer a way to run complete end-to-end models in an efficient manner; it is not an inference engine. TVM [10] takes the complementary route, scheduling from a computation description, and MLIR [11] provides the dialect infrastructure that several of these systems share.

2.2 Compilers that lower PyTorch

`torch.compile` [8] generates Triton for eager PyTorch graphs [7]. It does graph folding, and can handle Triton kernels in Torch modules since PyTorch 2.1. However, the compiled models cannot be run in inference runtimes like TensorRT and ONNX and run in Python, depending entirely on `libtorch`, which stops them from being efficiently usable in C++-only inference servers. AOTInductor [9] produces shared libraries, but still links the PyTorch C++ runtime. While not a deployment path, it is the only other way to compile torch models that contain Triton kernels, making it a very strong baseline for this paper.

2.3 Inference engines and custom operators

TensorRT [16] and ONNX [15] both expose C++ extension points (`nvinfer1::IPluginV2DynamicExt` with `IPluginCreator`; `OrtCustomOp`) and drive execution by themselves, handing operators raw device pointers. KernelLens complements this by avoiding the manual writing of those extensions and the rewriting of the Triton kernel to a CUDA kernel, often the hardest part. Serving systems such as vLLM [13] and TensorRT-LLM [14] take the same approach one level up, fusing standard layers and dispatching custom kernels from a C++ runtime.

2.4 Triton kernel libraries

Liger-Kernel [3], FlashAttention [5; 6] and AOTriton provide high-quality Triton kernels, many of them written for the model families they serve, such as Llama 3 [2] and Qwen 2.5 [4]. KernelLens is the deployment layer beneath such libraries, allowing them to be usable in an inference environment. It doesn't compete on kernel quality, it makes them deployable.

3 The Manual Pipeline

Before detailing the methodology, it is worth being concrete about what this replaces. Figure 1 displays a comparison between what a developer would have to build (on the top) and what they would need to do with KernelLens (on the bottom) to serve PyTorch modules containing Triton kernels. We detail in this section the two-step approach we took with KernelLens.

3.1 Compilation

As shown in Figure 1, serving the module would amount to:

(1) Port all the kernels to C++, one by one, debugging through invalid memory accesses, CUDA memory errors and different language behaviours.

(2) Implement the ONNX Runtime and TensorRT classes for all those kernels.

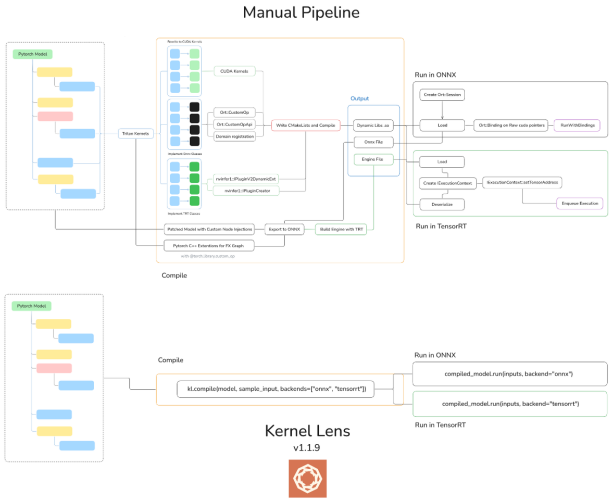


Figure 1. Top: the manual pipeline. Every box is a class, a build step or an engine call written and maintained by hand—rewriting the kernel in CUDA, `Ort::CustomOp`, `Ort::CustomOpApi`, domain registration, `IPluginV2DynamicExt`, `IPluginCreator`, CMake, FX-graph patching with `@torch.library.custom_op`, ONNX export, TensorRT engine building, `RunWithBindings` and `Enqueue Execution`. Bottom: KernelLens.

(3) Write CMakeLists and compile them.
(4) Patch the original model by replacing the Triton calls with custom node injections.
(5) Write the PyTorch C++ extensions for the FX graph, for each kernel.
(6) Export the model to ONNX.
(7) Reload the model with TRT and build the engine.

3.2 Running

Both engines run in a different way. **ONNX Runtime:** create an `Ort::Session`, load the library, bind raw CUDA pointers through `Ort::Binding`, then call `RunWithBindings`. **TensorRT:** deserialize the engine into an `IExecutionContext`, call `IExecutionContext::setTensorAddress` for every tensor, then enqueue.

Each step is a place where the ABI, the launch geometry, the memory placement or the numerics can diverge from the original kernel, making manual rewriting very error-prone.

4 Formalisation

A compiled Triton kernel actually carries more information than its Python signature would suggest at first glance. We draw up an abstraction of the contract through the following formalisation:

Component	Meaning	Mechanism in the engine
$\mathbf{P}_{\text{in}}$	read-only pointers	operator inputs, bound by address
$\mathbf{P}_{\text{out}}$	write-only pointers	operator outputs, engine-allocated
$\mathbf{P}_{\text{inplace}}$	read and written	input/output aliasing
$\mathbf{S}$	element strides	`int64` runtime arguments
$\mathbf{D}$	per-call scalars	CPU inputs / plugin members
$\mathbf{C}$	`tl.constexpr`	baked in PTX, absent from the ABI
$\mathbf{G}$	launch grid	C++ integer expression evaluated at run time

Table 1. The seven components of the contract and the engine mechanism each maps to.

$$\mathcal{K} = \langle \mathbf{P}_{\text{in}}, \mathbf{P}_{\text{out}}, \mathbf{P}_{\text{inplace}}, \mathbf{S}, \mathbf{D}, \mathbf{C}, \mathbf{G} \rangle, \tag{1}$$

where $\mathbf{P}_{\text{in}}$, $\mathbf{P}_{\text{out}}$ and $\mathbf{P}_{\text{inplace}}$ are read-only, write-only and read–write VRAM pointers; $\mathbf{S}$ are element strides; $\mathbf{D}$ are dynamic host scalars; $\mathbf{C}$ are `tl.constexpr` values baked into PTX; and $\mathbf{G}$ is the launch grid, computed from runtime dimensions.

None of those elements is recoverable from the Python signature alone. This is caused by Triton dropping unused parameters and re-bitwidthing others. The Python call actually contains the information on the launch grid, and the compiled artifact the launch parameters, so both are equally important. This is why KernelLens employs a dual-pass approach, recovering all the needed information for the contract in the form of a manifest.

5 Recovering the Contract

In this section we detail the dual-pass method, what it produces, and how the three techniques use that information to recover the contract.

5.1 The observations: two traces, one scan

The concrete trace. The model runs once on the GPU. Each launch yields a compiled PTX, the dynamic shared-memory size, the block size, and every argument's shape, strides and dtype. The shared-memory size is not redundant with the symbolic trace, as Triton's PTX declares it as `.extern .shared .align 16 .b8 global_smem[]`, an empty-bracket extern array that the launch has to size per CTA, since each CTA gets its own allocation.

The symbolic trace. The module is re-executed under `make_fx(tracing_mode="symbolic")` with fake tensors, so that it can extract the lambda over SymInt sizes.

It yields the grid as a symbolic expression in Python that will later be transpiled into C++. In case this trace diverges from the concrete one, the concrete manifests are used unchanged, which keeps the compilation deterministic. This trace exists to make the engine shape-generic.

The source scan. KernelLens walks the kernel source, recursing through `@triton.jit` helpers, and yields two sets: the `tl.load` targets and the `tl.store` targets. This allows labelling the entry parameters of the PTX call as either input, output or in-place.

The two traces and the source scan produce the `KernelManifest`, the structured form of the contract.

5.2 Technique (i): PTX entry-signature recovery

Inputs. The PTX from the concrete trace, and the argument array with types from the source scan.

Behaviour. Triton emits an `.entry` declaration that does not correspond to the Python signature: parameters have been dropped and some re-bitwidth'd. KernelLens parses it, determines each slot's C type and its textual parameter number, and maps the slots onto the manifest's arguments. The launch argument array is then rebuilt to match the PTX exactly, allowing the already compiled kernels to be run inside TensorRT or ONNX Runtime without a CUDA rewrite. Here is a detailed breakdown.

Extract the signature. Everything between `.entry` and the closing `.reqntid`. That slice is the ABI.

Split it into slots. Split on `.param`; per piece, the last token is the identifier and the rest is its declaration.

Number each slot from `_param_(\d+)` on the identifier, then index that number into the manifest's arguments and filter out the constexpr arguments.

Type each slot. We map each argument to its C++ type. We use the source scan recovery to map input/output/in-place pointers to `void*`.

Emit one array entry per slot. Each entry gets a link to a local pointer or scalar in the custom C++ class.

While this parsing recovers most arguments, two classes are absent from the ABI: constant parameters, which are baked into the PTX, and integer arguments Triton lowered to a constant. Both are reported as `constexpr` in the compiler's signature. The tracer collects those indices and marks the matching manifest arguments, so that the list used to number the arguments and the ABI end up with the same order. Table 2 lists what is recovered and from where.

Recovered	From
Parameter order, count and bitwidths	`.entry <kernel> (. param ...)` in the PTX
Entry function name (mangled)	`.visible .entry <name>`
Block size	`.reqntid`
Shared-memory size	the compiled kernel's metadata
Launch grid	the symbolic pass (Pass 2)
Input / output / in-place	the AST scan (Pass 3)

Table 2. Each component of the contract and the artifact it is recovered from.

5.3 Technique (ii): symbolic grid capture and C++ transpilation

Inputs. The captured dynamic grid lambda, a SymInt expression from the symbolic trace.

Behaviour. We capture the manipulated values in the PyTorch model as expressions of the inputs and parameters, and then transpile them into C++ arithmetic that can be evaluated at runtime in the plugins. KernelLens's translation from Python to C++ follows the steps below.

Build the symbol-to-dimension map. Using the FakeTensors' symbolic shapes, each symbol maps to a C++ read of the dimension it stands for. The backend supplies the accessor, so a single transpilation works for both engines (TensorRT and ONNX).

Expand powers before substituting. C++ doesn't express math the same way Python does and has no power operator. We expand the power to avoid using `std::pow`, since it drags in `cmath` and routes exact integer math through doubles. The same way, we simply replace Python floor division with the base C++ division and a cast.

Guard the uncovered case. If a symbol is not substituted properly, we emit it as a constant, making a static but still valid result.

One detail is worth stating because it is invisible: the expression has to be read from `SymNode._expr`, since `SymNode.expr` substitutes the values the shape environment has recorded and returns the collapsed constant — the correct answer for a shape-specialised engine, and exactly the wrong one for a shape-generic one.

5.4 Technique (iii): static argument taxonomy

Input. The load and store sets from the source scan.

Behaviour. TensorRT and ONNX both need to be told what each output is. ONNX Runtime allocates through `ctx.GetOutput(idx, dims)` with the manifest's dims and declares `GetOutputTypeCount`, and similar functions

exist for TensorRT. If an input or output is mislabeled, the memory needed will be invalid, leading to CUDA crashes at runtime.

Extracting the argument types. KernelLens inspects the source code by parsing the Triton AST, extracting `tl.store` and `tl.load` calls, then backtracking to extract the source pointers behind those calls through the graph: collecting the expression's free names, and following each locally bound one into its right-hand side until the function's parameters. This builds two sets, one of stores and one of loads.

Argument resolution. If an argument is in both sets it becomes in-place; store only makes it an output, and load only an input.

Why resolution above heuristics? The cheaper test might be "does the argument's name occur in the unparsed target?" It is actually a blind test, as a pointer can be renamed through assignment to an intermediate, and it over-matches short names. Resolution compares identities and fails at neither.

6 Re-establishing the Contract

Once the manifest has been recovered, KernelLens needs to convert it into working engine operators. The entrypoint for custom ops in TensorRT and ONNX Runtime differ a lot, and KernelLens generates the classes automatically.

What appears as a single line with KernelLens (`kl.compile(model, sample_input, backends=[...])`) is actually a multi-step pipeline (Figure 2).

Graph rewriting. A hook inside `torch.onnx.export` replaces each Triton launch with a custom node `triton_custom::kernel` carrying the PTX and the taxonomy. Per-call scalars become CPU inputs, so a new value needs no recompilation. The node is injected into the graph, so TensorRT and ONNX Runtime can still fold and tune the surrounding standard layers.

Plugin synthesis. For each kernel, KernelLens emits an `OrtCustomOp` class and a TensorRT plugin alongside its creator. Both reconstruct the launch argument array from the recovered ABI and evaluate the ported grid expression in C++ over the dimensions at runtime. This way, one compiled plugin serves dynamic shapes. The TensorRT plugin is emitted against both the V2 and the V3 APIs: TensorRT 10 deprecated the legacy `IPluginV2*` hierarchy and TensorRT 11 removed it, so a plugin built against the legacy interface alone cannot be loaded by a current runtime.

Build-time hardening. PTX ISA versions are clamped on load failure for older driver JITs, and a 16-byte alignment guard stages sub-aligned tensor slices through an aligned buffer.

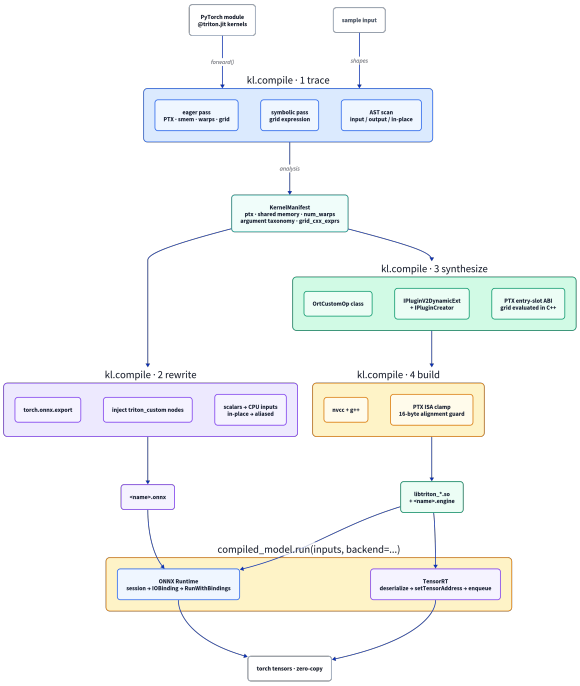


Figure 2. The compile pipeline. Trace recovers the contract into a `KernelManifest`; rewrite injects `triton_custom` nodes into the ONNX graph; synthesize emits the ONNX Runtime and TensorRT classes, rebuilding the PTX entry-slot ABI and evaluating the grid in C++; build compiles the libraries. The same artifacts drive both run lanes.

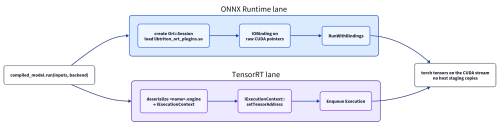


Figure 3. The run pipeline. One run call dispatches to either lane; neither moves tensor data through the host.

Run. `compiled_model.run(inputs, backend=...)` loads the library and hands PyTorch's GPU addresses straight to the engine (Figure 3). ONNX Runtime: create the session, bind raw CUDA pointers through `IOBinding`, `run_with_iobinding`. TensorRT: deserialize into an `IExecutionContext`, `set_tensor_address` for every tensor, `execute_async_v3`. Both cost $O(1)$ pointer registration plus the kernel, versus $O(N)$ host staging copies.

7 Implementation and Code Comparison

KernelLens is a Python package. The compiler is five modules—tracing, AST analysis, manifest assembly, ONNX export, and the two code generators—and the

runtime is a single `CompiledModel` that dispatches to ONNX Runtime or TensorRT. The user writes no C++.

The clearest way to convey what the framework does is to compare the two sides of the same task: one fused RMSNorm kernel, running as a native operator in both engines.

With KernelLens. Listing 1 is the entire deployment. The kernel and module are the ones the user already wrote.

```python
import kernel_lens as kl

# the Triton kernel + module you already wrote
compiled = kl.compile(
    RMSNorm(4096).cuda(), (x,),
    name="llama3_rmsnorm",
    backends=["onnx", "tensorrt"],
)
out_ort = compiled.run((x,), backend="onnx")
out_trt = compiled.run((x,), backend="tensorrt")
```

Listing 1. The KernelLens side: the entire deployment, nine lines.

Without KernelLens. Listing 2 is the shape of what you write instead: a CUDA port of the kernel, an `OrtCustomOp`, a TensorRT plugin and its creator, a build, a graph patch, and the engine-driving code.

```cpp
// 1. port the kernel to CUDA                          (rmsnorm.cu)
// 2. ONNX Runtime custom operator
struct RmsnormOp : Ort::CustomOpBase<...> {
    const char* GetName() const { return "rmsnorm"; }
    ONNXTensorElementDataType GetInputType(size_t i) const;
    // ... plus a Compute() that rebuilds the PTX argument array
};
// 3. TensorRT plugin + creator
class RmsnormPlugin : public nvinfer1::IPluginV2DynamicExt {
    DimsExprs getOutputDimensions(...) noexcept override;
    bool supportsFormatCombination(...) noexcept override;
    int32_t enqueue(...) noexcept override;          // launch
    size_t getSerializationSize() const noexcept override;
    void serialize(void*) const noexcept override;
    IPluginV2DynamicExt* clone() const noexcept override;
};
class RmsnormPluginCreator : public nvinfer1::IPluginCreator { /*
      ... */ };
// 4. CMakeLists.txt, nvcc, g++, header resolution
// 5. a PyTorch C++ extension that patches the FX graph
//    and wraps each kernel with @torch.library.custom_op
// 6. drive the engines
//    Ort::Session + Ort::Binding + RunWithBindings
//    IExecutionContext + setTensorAddress + Enqueue Execution
```

Listing 2. The manual side: classes, build and engine calls you own.

Measured volume. Table 3 counts the generated artifacts for this one kernel, compiled for both backends. It is 1,137 lines of C++ across six files—before CMake, the FX-graph patch, or the engine-driving code.

Artifact	Lines	Author
`ort_plugins/<kernel>Op.cu`	686	hand, without KernelLens
`ort_plugins/<kernel>Op.h`	60	hand
`ort_plugins/register_ops.cpp`	25	hand
`trt_plugins/<kernel>Plugin.h`	113	hand
`trt_plugins/<kernel>Plugin.cu`	230	hand
`trt_plugins/register_plugins.cpp`	23	hand
Total, one kernel	**1,137**	**KernelLens: 0**

Table 3. Generated C++ volume for a single fused RMSNorm kernel, both backends, measured with `tests/measure_codegen.py`.

Workload	ORT	TRT
LLaMA 3 / Mistral fused RMSNorm	0.00e+00	0.00e+00
Liger / PaLM SwiGLU	0.00e+00	0.00e+00
LLaMA 3 / Qwen 2.5 RoPE	0.00e+00	0.00e+00
Liger fused softmax + cross-entropy	0.00e+00	0.00e+00
Fused residual + RMSNorm	$\leq$ 9.5e−07	$\leq$ 9.5e−07
Mamba SSM recurrence	4.8e−07	4.8e−07
FP8 scaled GEMM / INT4 W4A16 GEMM	0.00e+00	—
Multi-output rank-4/3 log-sum-exp	0.00e+00	0.00e+00
In-place mutation (fp32 / fp16)	0.00e+00	0.00e+00

Table 4. Maximum absolute difference versus native Triton.

8 Evaluation

8.1 Experimental setup

All measurements are on a single NVIDIA GeForce RTX 5060 Ti (16 GB, Blackwell, `sm_120a`) with driver 570.144, CUDA 12.8, PyTorch 2.14.0, Triton 3.8.0, ONNX Runtime 1.30.0 and TensorRT 10.16.1.11. Timings use CUDA events after warm-up. Parity is measured against native Triton execution of the same kernel, which isolates the recovered plumbing from the arithmetic.

The conformance, parity and real-model results are additionally reproduced on a second architecture: an NVIDIA A100-SXM4-80GB (Ampere, `sm_80`) with driver 610.43.02, CUDA 13.3, PyTorch 2.12.1, Triton 3.7.1, ONNX Runtime 1.31.0 and TensorRT 10.16.1.11. Table 9 is measured on that configuration. Results are reported per configuration and never blended across them.

8.2 Numerical parity

Table 4 reports the maximum absolute difference between the plugin's output and the same kernel run natively in Triton. Six of the nine workloads are bit-exact on both backends, including the two fused attention-adjacent cases and the in-place mutations. Two land just above zero — fused residual plus RMSNorm at 9.5e−07 and the Mamba recurrence at 4.8e−07 — which are single-ULP-scale differences in fp32 accumulation, and they are reported rather than rounded to zero. The quantised GEMMs appear for ONNX Runtime only: neither is validated on the TensorRT path here. Parity is measured against native Triton of the *same* kernel, which isolates the plumbing from the arithmetic; it does not certify the kernel itself against an independent reference.

8.3 Coverage of module patterns

Parity on chosen operators is necessary but not sufficient: the recovery has to work for the ways a module actually calls a kernel. A conformance suite exercises fourteen patterns against both backends, all at 0.00e+00: two kernels in one graph; the same kernel launched twice with different scalars (each launch receives its own operator and plugin); multiple

Configuration	Latency (ms)	Relative
A. Folded only — eager, standard ops	28.04	0.54×
A. Folded only — `torch.compile`	15.15	1.00×
B. Kernels only — eager Triton	8.24	1.84×
B. Kernels only — `torch.compile`	7.55	2.01×
C. KernelLens — ONNX Runtime	9.27	1.63×
C. KernelLens — TensorRT	**7.63**	**1.99×**

Table 5. Two-layer block, D=512, H=8, S=4096, batch 1. Relative is versus the folded-only `torch.compile` row.

Operator	PyTorch Eager	torch.compile	Native Triton	KernelLens ORT
LLaMA 3 RMSNorm (4,512,4096)	0.517 ms	0.173 ms	**0.160** ms	0.172 ms
PaLM SwiGLU (2048×4096)	0.429 ms	0.252 ms	**0.252** ms	0.270 ms
Qwen 2.5 RoPE (4,512,32,128)	0.673 ms	0.174 ms	**0.159** ms	0.180 ms
Fused cross-entropy (32,32000)	0.013 ms	0.013 ms	**0.010** ms	0.026 ms

Table 6. Single-operator latency. The C++ operator runs the same kernel within 6–13% of native Triton on the larger operators; the value is deployment.

outputs of different shapes; dynamic host scalars; strided and dynamic-shape inputs; kernels inside a Python loop and behind a conditional; kernels delegating to nested helpers; in-place mutation; a 3-D launch grid; and fp16 and bf16 tensors. The one boundary is dtype coverage on ONNX Runtime, which cannot bind a bfloat16 tensor to a custom operator; bf16 models use TensorRT, which handles them exactly.

8.4 Folding plus kernels

This is the payoff of injecting the operator *into* the graph. We take a two-layer block whose attention is a custom squared-ReLU flash kernel, whose normalisations and gated activation are custom kernels, and whose projections and residuals are standard operators. It can be run three ways.

KernelLens runs the block in 7.63 ms: $1.99\times$ faster than the folded-only formulation and $3.68\times$ faster than eager folding, because folding has to materialise the $O(N^2)$ attention that the kernel avoids; and $1.08\times$ faster than the unfused kernel path, because the engine folds the standard layers around it. The two halves are complementary, and KernelLens is the only configuration that has both.

8.5 Single-operator latency

Table 6 brackets the C++ operator between three baselines on four operators. `torch.compile` is the meaningful comparison, because it captures the same Triton kernels rather than reimplementing them, and it is the column the plugin must beat to be worth its boundary. On the three larger operators the plugin runs within 6–13% of native Triton — the residual is the plugin call and argument marshalling, not the kernel, which is unchanged. The smallest operator is the clearest illustration of that distinction: fused cross-entropy is 0.010 ms natively and 0.026 ms through the plugin, because at that size the launch dominates the work.

Configuration	TTFT (S=512)	ITL (S=1)	Total (128 tok)	Peak VRAM
PyTorch Eager + Triton	7.85 ms	0.866 ms	0.118 s	478.0 MB
torch.compile (Inductor)	7.92 ms	0.866 ms	0.118 s	473.0 MB
KernelLens C++ plugins (ORT)	4.37 ms	0.999 ms	0.131 s	464.2 MB
KernelLens TensorRT plugin	**4.14** ms	0.905 ms	0.119 s	466.2 MB

Table 7. End-to-end 4-layer LLaMA decoder (D=1024). The TensorRT plugin nearly halves time-to-first-token and uses less peak VRAM than either PyTorch configuration.

Kernel	Eager FP16	Native Triton	KL ORT	Speedup
FP8 scaled GEMM (4096×4096)	2.786 ms	2.046 ms	2.083 ms	1.34×
INT4 W4A16 GEMM (2048×4096)	1.388 ms	3.570 ms	3.629 ms	0.38× (2× smaller)

Table 8. Quantized GEMMs on `sm_120a`, where PTX ISA clamping is exercised. Both reach exact parity with native Triton.

8.6 End-to-end

Table 7 runs a four-layer LLaMA decoder end to end, which is where the single-operator results have to add up to something. Replacing the eager Triton kernels with the TensorRT plugin nearly halves time-to-first-token, from 7.85 to 4.14 ms, and uses less peak VRAM than either PyTorch configuration. The shape of the result is prefill: prefill is where the fused graphs pay off, and inter-token latency is correspondingly almost flat across all four configurations — the ONNX Runtime path is in fact marginally worse than eager (0.999 vs 0.866 ms) while the TensorRT plugin holds 0.905 ms. Total time for 128 tokens is therefore essentially unchanged. We report it this way rather than as a single speedup, because the win is in prefill and not in decode.

8.7 Quantized kernels

Table 8 covers the two quantised GEMMs on `sm_120a`, the host where the PTX ISA clamping path is actually exercised. Both reach exact parity with native Triton. FP8 scaled GEMM is $1.34\times$ faster than eager fp16, which is the expected direction. INT4 W4A16 is slower than native Triton ($0.38\times$) while halving the weight footprint; that is a deliberate trade a serving system makes for capacity rather than a defect of the plumbing, and it is reported per kernel rather than blended into a single quantisation figure.

8.8 Real pretrained models

The models above are controlled PyTorch modules, which keeps the plumbing comparable but leaves open whether the approach scales to a model people actually serve. We therefore take real pretrained OPT checkpoints, replace every `LayerNorm` in the decoder with the Triton RMSNorm kernel above (carrying the weight across), and compile the result. OPT is used because its architecture is exportable through the TorchScript path the compiler relies on; the rotary-embedding families are not, and that is a property of the ONNX exporter rather than of the recovered contract, since plain `torch.onnx.export` fails on them identically.

TensorRT is the fastest configuration throughout, but the

Model	dtype	Eager	`torch.compile`	ORT	TensorRT
OPT-125M	fp32	12.1 ms	10.9 ms	4.8 ms	**3.4** ms
OPT-125M	fp16	5.7 ms	2.4 ms	3.6 ms	**2.0** ms
OPT-1.3B	fp32	98.6 ms	88.0 ms	23.4 ms	**19.0** ms
OPT-1.3B	fp16	14.7 ms	10.8 ms	15.6 ms	**9.9** ms
OPT-2.7B	fp32	175.9 ms	172.7 ms	37.4 ms	**31.7** ms
OPT-2.7B	fp16	22.2 ms	17.5 ms	22.6 ms	**16.5** ms
OPT-6.7B	fp16	37.0 ms	33.8 ms	40.2 ms	**31.1** ms

Table 9. Real pretrained OPT checkpoints with every decoder `LayerNorm` replaced by the Triton RMSNorm kernel. Prefill latency, S=512, on an A100-SXM4-80GB. `torch.compile` is the meaningful baseline: it captures the same kernels rather than reimplementing them. Bold marks the fastest configuration, which is TensorRT at every size and precision.

comparison that matters is against `torch.compile` rather than eager, since it captures the same Triton kernels and differs only in how the surrounding operators are scheduled. The picture splits by precision.

In fp32 `torch.compile` barely improves on eager (1.02–1.12×) because it does not fuse the operators around the kernel, whereas KernelLens does: TensorRT is 3.18×, 4.62× and 5.45× faster than `torch.compile` on 125M, 1.3B and 2.7B, and the margin grows with model size. That growth is the graph-level fusion at work, not the normalisation kernels.

In fp16 the ordering reverses. `torch.compile` then optimises well (2.4× over eager at 125M), and KernelLens only matches it: TensorRT edges `torch.compile` by 1.06–1.22× across all four sizes, while ONNX Runtime is consistently *slower* (0.67–0.84×). The compiled fp16 graph for OPT-125M carries 1,583 nodes (76 `Cast`, 345 `Unsqueeze`, 549 `Constant`) against 25 custom operators, so per-node dispatch overhead dominates at these shapes.

Keeping kernels therefore pays only when the surrounding graph is left unfused. When the baseline already fuses, it breaks even, and we do not claim otherwise.

Numerical agreement tracks precision rather than speed. In fp32 every model retains the same argmax on at least 98.8% of positions and the mean absolute logit difference stays below 6.3×10^{-3}; OPT-350M in fp16 under ONNX Runtime is bit-exact. Under TensorRT in fp16, agreement falls to 93.4–97.9% with a mean difference of up to 5.3×10^{-2}.

8.9 Shape-generic engines

Everything above measures a shape-specialised engine: compiled for one shape, fastest at that shape. The same plugin machinery supports the opposite trade-off, and TensorRT is not the obstacle—`IPluginV2DynamicExt`, the interface we emit, exists precisely for plugins inside dynamic-shape engines, and the optimization-profile API is standard.

```
kl.compile(..., dynamic_shapes=True)
```

Case	Compiled at	Served	Max diff
`dynamic_batch_sweep`	batch 8	1, 2, 4, 8, 16	`0.00e+00`
`dynamic_grid_sweep`	524,288	65,536 … 1,048,576	`0.00e+00`

Table 10. One engine per row, serving every shape listed, bit-exact against native Triton on both ONNX Runtime and TensorRT, on an RTX 5060 Ti (`sm_120a`) and an A100-SXM4-80GB (`sm_80`).

exports the leading dimension as a symbolic axis, which makes the network carry a `-1` dim and the profile apply. The generated plugins then evaluate their launch grid and their shape-derived scalars from the tensor at run time rather than from the traced shape. `dynamic_max` bounds the profile; it defaults to 4× the traced size, because TensorRT compiles kernels spanning `min...max` and a needlessly wide range costs build time and can cost per-shape performance.

Table 10 reports the two conformance cases that exercise the capability rather than assuming it: each compiles once and then runs every shape through that single engine, on both backends.

Two defects had to be fixed for this to work, and both are instructive because neither is visible under shape specialisation. The first is the accessor pitfall described in Section 5: reading `.expr` instead of `._expr` collapses the symbolic grid to the traced constant. The second is in how the plugin's PTX is obtained. We recover it by walking Triton's compile cache, which holds one variant per argument specialisation, keyed by the argument types:

```
[('*fp32', 'D'), ..., ('constexpr', 128)]
```

Selecting a variant by cache position rather than by that key is wrong, because a re-used entry keeps its original slot: an `fp32` variant compiled early sits *before* a `bf16` one compiled later, and the `fp32` launch is then handed the `bf16` PTX. That kernel reads `fp32` memory 16 bits at a time and silently covers half the tensor—correct at the traced shape, and wrong everywhere else. Matching the variant against the launch's own argument types fixes it.

8.10 Spanning the TensorRT plugin API generations

Because TensorRT 10 deprecated the `IPluginV2*` hierarchy and TensorRT 11 removed it (Section 6), a plugin written only against the legacy interface cannot be loaded by a current runtime at all. If the recovered contract were an artifact of one plugin API generation, the approach would inherit that generation's lifetime. It does not: the same manifest drives both emitters, and both are measured.

Table 11 reports the conformance suite — sixteen cases spanning the recovery techniques of Section 5: strided access, multiple kernels in one build, kernels with two outputs, in-place arguments, 3-D grids, dynamic scalars, and dynamic batch and grid sweeps. Each row is a different runtime and

TensorRT	Headers	plugin_api	Cases
10.16.1.11	bundled	v3 (auto)	**16/16**
10.16.1.11	bundled	v2 (forced)	**16/16**
11.3.0.99	release/11.3	v3 (auto)	**16/16**
10.14.1.48	bundled	v3 (auto)	**16/16**

Table 11. Conformance across TensorRT versions and both plugin API generations. TensorRT 11 has no IPluginV2* interface to emit against, so the 11.3 row is only reachable through the IPluginV3 path; the forced-v2 row shows the legacy generation is not merely retained but exercised, so the two paths are compared rather than one assumed. The 10.14.1.48 row is an Ampere host (sm_80) running the same suite through ONNX Runtime 1.20.2. All four runs are 16 of 16.

a different emitted interface, and all four are complete.

Two further checks support the same reading. Forcing plugin_api="v2" on TensorRT 10 produces byte-identical generated sources to the pre-refactor baseline, so the shared emitter had not silently drifted; and the parity suites, which compare plugin output against native Triton, return values identical across the two emitters — 0.00e+00 on the research kernels, the in-place and quantised GEMM cases and the edge cases, 9.54e−07 on residual plus RMSNorm and 4.77e−07 on Mamba. The interface KernelLens recovers is therefore re-established, not approximated, and it is re-established on an interface generation chosen from the environment rather than fixed in the paper.

9 Discussion

When the engine wins, and when it does not. TensorRT is not universally faster than torch.compile, and we report both directions. On GEMM-heavy transformer prefill the TensorRT plugin is $1.9\times$ faster (4.14 versus 7.92 ms). On a convolution-heavy graph with a custom kernel it is behind, even after fixing a dtype-promotion bug and declaring channel-last formats for pointwise kernels; the remaining gap is the engine's convolution performance rather than the plugin.

Why an engine at all. The reasons are structural as much as numerical: no Python interpreter and no libtorch in the serving path (no GIL, a small footprint, deployable from non-Python services); a serialized engine that cold-starts without recompiling; engine-level VRAM planning (464 MB versus 473/478 MB in our decoder); dynamic shapes handled by optimisation profiles rather than recompiles and guards; and quantized inference, where TensorRT is mature. The folding result is the numerical case: if the kernel cannot be shipped at all, the folded-only formulation is $2\times$ slower.

10 Threats to Validity

Internal validity. All timings are CUDA-event measurements on one machine and one GPU. The folding comparison is sensitive to precision: an early version of it showed a larger gap that was mostly TensorRT's default TF32 rather than folding, and we enable TF32 in the PyTorch baselines to remove that confound. Parity is measured against native Triton of the same kernel, which isolates the plumbing from the arithmetic but does not certify the kernel itself against a reference implementation.

External validity. The evaluation spans two architectures (Blackwell consumer and Ampere datacenter) but a single vendor, a single driver per architecture, and fp32/fp16/bf16/FP8/INT4 on Blackwell. The absolute numbers do not transfer to other GPUs or to distributed serving, and the "engine wins on prefill" conclusion is specific to the shapes tested.

Construct validity. "Coverage" is demonstrated by fourteen module patterns, which is a strong sample but not a proof of universality; genuinely exotic cases (custom autograd.Function kernels, CUDA-graph capture, collectives inside a kernel's forward) are untested.

11 Limitations and Future Work

ONNX Runtime cannot bind bfloat16 to a custom operator, so bf16 models use TensorRT. TensorRT's tensor path required routing INT8 and in-place-aliased operators to the exact ONNX Runtime plugin. The generated ONNX Runtime registrar exports a fixed symbol, so several compiled models that share a kernel base name can interfere when loaded in one process. Future work: auto-tuning dispatch (folding @triton.autotune variants into one plugin with a shape-keyed table); CUDA-graph capture of the launch sequence; channel-last formats for non-pointwise kernels; distributed execution with embedded NCCL handles; and a Blackwell backend (cluster-launch descriptors, cp.async.bulk, NVFP4).

Engine coverage beyond ONNX. The recovered contract is engine-agnostic: it describes how a compiled Triton kernel maps onto an engine's tensors, and only the injection step is engine-specific. TensorRT-LLM is a natural next target, being the standard way to serve LLMs on NVIDIA hardware. The plugin is not the obstacle: its plugins are TensorRT plugins, and its PyTorch backend additionally accepts torch.library custom operators, so a kernel can reach either path. The work is in the injection, which is graph-level and has no equivalent here, because TensorRT-LLM builds from a checkpoint or its own module API rather than from ONNX. Scope would be non-attention kernels, since

its attention plugin owns the paged KV cache and in-flight batching, a contract a user kernel cannot re-enter.

12 Conclusion

A compiled Triton kernel is a self-contained artifact, but the interface between it and a C++ engine is implicit. We showed that the interface can be recovered automatically—the PTX entry signature, the symbolic launch grid, and the argument taxonomy—and re-established as native TensorRT and ONNX Runtime operators injected into the graph. The composition is exact on the research kernels and fourteen module patterns, it replaces 1,137 lines of hand-written C++ per kernel with nine lines of Python, and it pays off: keeping the kernel while letting the engine fold the rest is $1.99\times$ faster than the folded-only formulation and $1.08\times$ faster than the unfused kernel path.

13 Reproducibility Statement

KernelLens is released as a Python package on PyPI, and the complete source tree, the evaluation scripts and every raw measurement are committed in the repository. The accompanying appendix records the provenance of every number reported here: the full configuration of both machines, the measurement procedure behind each family of results, the definition of the fourteen conformance patterns and the two shape sweeps, the counting rule behind the code-volume figure, and the recorded values at full precision — including the second-architecture runs and the outcomes that do not favour the approach. The evaluation scripts are grouped by what they establish: a conformance suite, a parity suite over the research kernels, benchmark drivers for the folding comparison, single-operator latency and end-to-end runs, quantised GEMM tests, and the code-volume measurement. Each writes its results as JSON alongside the data it is derived from, and those files are the source of every value. The implementation is prepared for the artifact evaluation process and will be released publicly with the paper. All results were produced on the RTX 5060 Ti configuration described in §8, with the second-architecture results produced on an A100-SXM4-80GB.

References

[1] P. Tillet, H. Kung, and D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," in *ARRAY*, 2019.

[2] AI@Meta, "The Llama 3 Herd of Models," *arXiv:2407.21783*, 2024.

[3] LinkedIn Corp., "Liger-Kernel: Efficient Triton Kernels for LLM Training," 2024.

[4] Qwen Team, "Qwen2.5 Technical Report," *arXiv:2412.15115*, 2024.

[5] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *NeurIPS*, 2022.

[6] T. Dao, "FlashAttention-2," in *ICLR*, 2024.

[7] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *NeurIPS*, 2019.

[8] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. K. Luk, B. Maher, Y. Pan, C. Puhrsch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews, G. Chanan, P. Wu, and S. Chintala, "PyTorch 2.0: TorchDynamo and TorchInductor," in *ASPLOS*, 2024.

[9] PyTorch Engineering Team, "PyTorch AOTInductor," 2023.

[10] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning," in *OSDI*, 2018.

[11] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "MLIR: Scaling Compiler Infrastructure for Domain Specific Architectures," in *CGO*, 2021.

[12] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: A Language and Compiler for Optimizing Parallel Image Processing Pipelines," in *PLDI*, 2013.

[13] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for LLM Serving with PagedAttention," in *SOSP*, 2023.

[14] NVIDIA Corporation, "TensorRT-LLM," https://developer.nvidia.com/tensorrt, 2023.

[15] Microsoft Corporation, "ONNX Runtime," https://onnxruntime.ai, 2021.

[16] NVIDIA Corporation, "NVIDIA TensorRT," https://developer.nvidia.com/tensorrt, 2023.

AI USAGE DISCLOSURE

An AI coding assistant (DeepSeek V4.1 Flash, driven through OpenCode) was used in preparing this work, and the author takes full responsibility for all content. Its contributions were: (i) drafting and editing the manuscript text; (ii) debugging the `kernel_lens` package and implementing its generic shape system; (iii) implementing the V3 API support on the basis of the existing V2 code path; (iv) writing a minor portion of the regression tests; (v) producing the diagrams and result figures, with the exception of Figure 1, which is the author's; and (vi) executing the benchmarks on the GPU host and tabulating the measurements.

The intellectual content of the work is the author's: the design and implementation of the package, the experimental design, the literature search and the interpretation of the results were performed by the author, and no AI tool was used for ideation. No result, citation or number was generated by a model: every measurement reported here was produced by executing the code on real hardware, and the author has verified all claims, references and measurements.